多层感知机理论

Part 1：概念解释

1. 基础结构

- 隐藏层

> 指的是输入层与输出层中间的层数，称为中间层或隐藏层，输入层传入的数据乘以权重才加上偏置进入隐藏层后再经过激活函数最后输出到下一层。

- 全连接层

> **当前层的每一个神经元，都和上一层的所有神经元全部相连**，所以叫“全连接”，它只做**线性变换**（默认**不带激活函数**）。

- 激活函数（Sigmoid、Tanh、ReLU、Leaky ReLU、Softmax）

> 指中间层在接收到输入数据后通过一个指定函数变换，将输入信号的总和转换为输出信号。

- 权重矩阵与偏置

> 指层与层之间信息交换传输所加入的权值，如$y = wx + b$，w为权重，b为偏置，**权重是控制输入信号重要性的参数，偏置是调整神经元被激活容易程度的参数。**

2. 训练机制：

- 前向传播

> 指数据从输入层经过每一个隐藏层的权重偏置变换再经过激活函数最后到输出层这一完整过程。

- 反向传播

> 指经过了一轮前向传播后输出层结果与真实结果比对计算损失和梯度后一步一步向前传输并同步更新权重与偏置的过程。

- 损失函数（MSE、Binary Cross Entropy、Cross Entropy）

> 一种计算真实值与预测值之间差距的函数，神经网络的学习就是通过损失函数当前的状态并以该函数为基准寻找最优参数。

- 梯度下降与优化器（SGD、SGD+Momentum、Adam）

> 梯度下降指每次的参数都向损失函数在当前位置的梯度方向更新，以达到最快优化参数的目的；优化器是在梯度下降基础上增加各种技巧，就形成不同优化器。
>
> > 随机梯度下降 SGD：只用 1 个样本计算梯度，立刻更新
> >
> > SGD+Momentum：在SGD基础上保留历史的移动方向即动量，相当于给小球添加惯性一样

> > Adam：同时带惯性 + 自适应学习率，收敛速度通常比普通 SGD 快很多，对不同参数自适应步长，不用精细调学习率，能处理稀疏梯度（Adam询问了ai）

3. 常见问题与对策：

- 过拟合、欠拟合

> 欠拟合：模型能力太弱，连训练集都学不好。
>
> > 对策：增加网络层数 / 神经元；换更复杂模型；延长训练轮数
>
> 过拟合：模型太强，只记住了训练数据的细节，**泛化差**。
>
> > 对策：增加训练数据；L1/L2 正则（权重衰减）；早停（提前结束训练）；简化模型，减少参数量

- 梯度消失、梯度爆炸

> 梯度消失：多层求导后，梯度不断乘上小于 1 的数，梯度变得极小，几乎接近 0。
>
> 梯度爆炸：多层求导，梯度不断乘大于 1 的数，梯度变得巨大。 对策：梯度裁剪；更换激活函数；BatchNorm；合理权重初始化。

- 暂退法（Dropout）

> 训练的时候，随机把一部分神经元输出置 0，让神经元不要依赖少数几个特定神经元。

- 权重衰减（L1 / L2 正则化）

> 在损失函数上额外加一项惩罚项，限制权重不要变得太大，用来抑制过拟合。 L2 正则（最常用，权重衰减）：惩罚项 $(\lambda \sum w^2)$，让权重趋向变小，不会爆炸。
>
> L1 正则：惩罚项$(\lambda \sum |w|)$，会把很多权重压缩到 0，实现权重稀疏化（很多参数直接等于 0）。

- 批归一化（Batch Normalization）

> 对一个 batch 内的特征做标准化：把特征拉到均值为 0，方差为 1。可减轻梯度消失 / 爆炸，允许用更大学习率，加速收敛

- 权重初始化（Xavier、He 初始化）（这一部分询问了ai）
  - **Xavier 初始化**：适合 Sigmoid/Tanh。保证**前向传播时，每层方差不变**，避免信号放大 / 缩小。
  - **He（何恺明）初始化**：适合 ReLU 及其变体。ReLU 会把一半值置 0，方差会减半，He 专门适配 ReLU，防止梯度消失。
- Dead ReLU 问题

> 指限于ReLU的特性当神经元输入数据一直为负数时神经元输出结果永远为0，即无法被激活，导致Dead ReLU

Part 2：回答问题

1. XOR问题与MLP的意义

（1）为什么单层感知机（线性模型）无法解决XOR异或问题？几何角度分析

XOR异或真值表：

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

单层感知机属于线性模型，决策边界是二维平面上的一条直线。 样本分布：正样本点$(0,1),(1,0)$；负样本点$(0,0),(1,1)$。几何上不存在一条直线，可以将正负样本完全分割开。 单层感知机只能解决线性可分问题，XOR属于线性不可分问题，因此单层感知机无法完成分类。

（2）MLP是如何解决XOR问题的？最少需要几个隐藏层、几个隐藏神经元？

MLP引入隐藏层 + 非线性激活函数，不再是单纯线性变换，可以构造多条直线组合，形成非线性的决策边界，从而分割线性不可分样本。 最小配置：1个隐藏层，2个隐藏神经元。 两个隐藏神经元各自生成一条分界线，经过非线性激活，两层线性变换组合，构造出非线性边界，完成XOR分类。

2. 参数与计算量分析

给定MLP：输入维度 d，一个隐藏层维度 h，输出维度 o，每层带偏置 b。

(1) 该网络共有多少可训练参数？（写出公式，注意偏置项）

- 隐藏层权重：$d \times h$；隐藏层偏置：h
- 输出层权重：$h \times o$；输出层偏置：o

$$\text{总参数数量} = d \cdot h + h + h \cdot o + o = h(d+1) + o(h+1)$$

(2) 参数以Float32存储占多少内存？INT8存储呢？

- Float32：单个参数占用4字节

$$\text{内存大小（Float32）} = 4 \times [h(d+1) + o(h+1)] \quad \text{Byte}$$

- INT8：单个参数占用1字节

$$\text{内存大小（INT8）} = 1 \times [h(d+1) + o(h+1)] \quad \text{Byte}$$

(3) 前向传播的计算复杂度（乘加运算MAC次数），写出表达式

偏置只做加法，不计入乘加运算。

- 输入→隐藏层：$d \cdot h$ 次乘加
- 隐藏层→输出层：$h \cdot o$ 次乘加

$$\text{总MAC次数} = d \cdot h + h \cdot o$$

3. 激活函数对比：ReLU vs Sigmoid vs Tanh

(1) 从梯度特性角度对比三者的优缺点

1. Sigmoid：$\sigma(x) = \frac{1}{1+e^{-x}}$
   - 优点：输出范围$(0, 1)$，可以近似表示概率；函数平滑可导。
   - 缺点：当$|x|$很大时函数饱和，梯度趋近0，易发生梯度消失；输出恒大于0，会让后一层输入偏移，收敛变慢。
2. Tanh：$\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
   - 优点：输出范围$[-1, 1]$，收敛速度优于Sigmoid；中间区间梯度较大。
   - 缺点：$|x|$很大时依旧饱和，梯度趋近0，深层网络仍然存在梯度消失问题。
3. ReLU：$\max(0, x)$
   - 优点：$x > 0$时梯度恒等于1，不会饱和，有效缓解梯度消失；无指数运算，计算速度快。
   - 缺点：$x < 0$时梯度直接为0，会产生Dead ReLU；输出非零均值。

(2) 什么是Dead ReLU问题？如何产生？如何缓解？

Dead ReLU：训练过程中部分神经元，输入始终小于0，输出恒为0，梯度一直为0，权重不再更新，神经元永久失效。

- 成因：学习率设置过大，一次参数更新后，神经元输入长期落在负数区间；权重初始化不当。
- 缓解方案：
  1. 使用He初始化；
  2. 调小学习率；
  3. 更换激活函数：LeakyReLU，在负区间保留很小的梯度。

(3) Sigmoid涉及指数运算，前向传播和反向传播常用什么方法简化计算？（此处询问了ai）

- 前向传播：使用**查找表LUT**，预先计算并保存一系列x对应的sigmoid值，推理时直接查表，避免实时计算指数。
- 反向传播：利用导数恒等式 $\sigma'(x) = \sigma(x)(1 - \sigma(x))$，复用前向已经计算得到的$\sigma(x)$，不需要重复计算指数，降低计算开销。

4. 暂退法（Dropout）如何防止过拟合？

(1) 从集成学习Ensemble 和特征共适应角度解释Dropout工作原理

1. **特征共适应（co-adaptation）**：过拟合时神经元之间互相依赖，神经元只在特定同伴神经元同时存在时才生效，学到训练集噪声，泛化能力差。Dropout训练时随机屏蔽一部分神经元，**阻止神经元相互依赖，迫使每个神经元学习独立鲁棒的特征**。
2. **集成学习角度**：每一轮前向传播，随机丢弃神经元，等价于每次训练一个结构不同的子网络。大量迭代等价于同时训练海量不同子网络；测试阶段相当于把所有子网络做集成平均，降低模型方差，抑制过拟合。

(2) 为什么训练时使用Dropout，测试时需要关闭它？

训练阶段随机丢弃神经元，会让输出期望变小。 测试阶段不可以随机丢弃神经元，否则输出分布不一致，预测结果不稳定，期望产生偏移。为保证训练和测试时输出期望保持一致，测试时需关闭，训练时启用。

Part5 深入探索实验报告

1. 深度与宽度：增加深度 vs 增加宽度对模型效果的影响

实验思路

- **宽度**：网络同一隐藏层内神经元数量
- **深度**：网络隐藏层的层数

对照组：基础模型（1层隐藏层，16神经元，ReLU，Adam） 实验组1（加宽）：1层隐藏层，32神经元 实验组2（加深）：2层隐藏层，每层16神经元

```
# 加宽模型
class MLP_Wide(nn.Module):
    def __init__(self, input_size=2, hidden_size=32, output_size=1):
        super().__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.fc2 = nn.Linear(hidden_size, output_size)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()
    def forward(self,x):
        x = self.relu(self.fc1(x))
        x = self.sigmoid(self.fc2(x))
        return x

# 加深模型
class MLP_Deep(nn.Module):
    def __init__(self, input_size, h1, h2, output_size):
        super().__init__()
        self.fc1 = nn.Linear(input_size, h1)
        self.fc2 = nn.Linear(h1, h2)
        self.fc3 = nn.Linear(h2, output_size)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()
    def forward(self,x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.sigmoid(self.fc3(x))
        return x
```

实验现象与结论

1. **增加宽度（同层数，增加神经元）** 模型拟合能力高，梯度消失风险减弱，但神经元过多容易学习噪声导致过拟合问题。

2. **增加深度（增加隐藏层，每层神经元不变）** 增加隐藏层过后模型层数增加，可以分层次提取抽象特征，便于拟合，但层数过多在反向传播的过程中多次求导容易造成梯度消失等问题。

2. 激活函数对比：ReLU / Sigmoid / Tanh，观察梯度消失

保持网络结构不变，仅替换**隐藏层激活函数**，观察梯度消失现象。

```
# 隐藏层Sigmoid版本
self.act = nn.Sigmoid()
# 隐藏层Tanh版本
self.act = nn.Tanh()
```

实验现象与结论

1. **ReLU** 训练收敛速度快，损失快速下降，最终测试准确率高。当输入大于0时梯度恒等于1，多层反向传播时梯度不会连续衰减，基本不会发生梯度消失，是隐藏层首选。
2. **隐藏层使用Sigmoid** 收敛非常缓慢，损失下降困难，最终准确率偏低。Sigmoid在两端饱和，最大梯度仅0.25；多层反向传播时梯度不断相乘，梯度消失现象明显，深层权重几乎得不到更新。
3. **隐藏层使用Tanh** 效果介于ReLU与Sigmoid之间，收敛速度优于Sigmoid，但弱于ReLU。Tanh梯度最大值为1，但两端依然存在饱和区，多层网络下依然会出现梯度消失问题，但程度轻于Sigmoid。

> 将隐藏层ReLU替换为Sigmoid，会明显观察梯度消失；替换为Tanh也会出现梯度消失，但程度更轻。ReLU可以有效缓解深层网络梯度消失。

3. 过拟合与正则化：Dropout 与 L2正则

构建大容量模型，使其发生过拟合；再分别增加Dropout、L2正则，观察测试集准确率变化。

```
# 大容量易过拟合模型 + Dropout
class MLP_Overfit(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(2, 128)
        self.fc2 = nn.Linear(128,128)
        self.fc3 = nn.Linear(128,1)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()
       # self.dropout = nn.Dropout(0.2)  训练时随机丢弃20%神经元
    def forward(self,x):
        x = self.relu(self.fc1(x))
      # x = self.dropout(x)
        x = self.relu(self.fc2(x))
      # x = self.dropout(x)
        x = self.sigmoid(self.fc3(x))
        return x

# L2正则写法（优化器参数weight_decay）
optimizer = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=1e-4)
```

实验现象与结论

1. 大容量模型：训练集准确率很高，测试集准确率明显低于训练集，出现**过拟合**。决策边界过度扭曲，学习到数据中的噪声，泛化能力差。
2. Dropout：**训练阶段**随机关闭部分神经元，阻止神经元之间过度依赖，降低模型对噪声的记忆。加入Dropout后，**测试集准确率上升，有效缓解过拟合**。
3. L2正则：对权重大小施加惩罚，限制权重不能无限增大，让模型权重更平滑，同样抑制过拟合，提升测试集泛化性能。

> Dropout在**网络结构层面**随机抑制神经元；L2正则在**损失函数**层面惩罚权重，二者都用来降低模型复杂度，缓解过拟合。

---

4. 优化器对比：SGD、SGD+Momentum、Adam

保持网络结构、学习率不变，仅更换优化器，对比收敛速度、损失曲线、最终准确率。

```
# 1. SGD 原始随机梯度下降
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# 2. SGD + Momentum 带动量SGD
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

# 3. Adam
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

实验现象与结论

优化器	收敛速度	特点
SGD	最慢	单纯沿当前梯度方向下降，损失震荡明显，容易卡在局部极小点，对学习率敏感
SGD+Momentum	中等	引入动量，累积历史梯度，减少震荡，加快收敛，相当于给梯度下降增加惯性
Adam	最快	融合动量与RMSprop自适应学习率，前期损失快速下降，对学习率鲁棒性更好

1. SGD：收敛最慢，震荡明显，需要仔细调参。
2. SGD+Momentum：利用历史梯度惯性，收敛速度提升，震荡减弱。
3. Adam：收敛速度最快，自适应调整每个参数的学习率，本双月数据集上训练效果最好；缺点是部分场景泛化性能可能弱于SGD。

关于 AI 使用

在整个ml-2任务中我多处使用了ai，具体如下：

1. part1 我在优化器方面不太了解，询问了ai关于Adam优化器的相关知识，对L1L2的区别不太了解，询问了对应的表达式，在Xavier、He初始化方面不太了解，询问了每个的含义。

2. part2 在简化sigmoid指数运算方面询问了ai对应的方法，在特征共适应和集成学习方面询问了ai。
3. part3/4 大部分的part3/4代码都是由ai所写的，然后我再询问它每行的含义和作用并在对应行加上注释便于自己理解（主要是手搓一个MLP确实能力有限= =再加上自己对pytorch神经网络构建还不太熟悉于是只能用ai写了）。
4. part5 询问了aiL2正则的写法，Dropout函数的写法以及SGD带动量的写法，对每个实验都询问了ai对应的实验思路以及该从哪方面着手。